

THE OPEN SOURCE AUDIT

A Capstone Project Report

Course: Open Source Software | Units 1–5 | Max Marks: 100

Software Chosen:

**Python — General Purpose Programming Language | Licence:
PSF Licence**

Student Name	NUPUR
Registration No.	24BCE10897
Slot	A24
Date of Submission	31/3/2026

1. Introduction

This report dives into Python as a prime example of open-source software success. It's one of the world's most popular programming languages—a high-level, general-purpose tool that's super readable and easy to pick up. Python powers everything from web apps and data analysis to AI, automation, and cutting-edge scientific work. Beginners use it for their first "Hello World," while pros at places like Google, NASA, and CERN rely on it for heavy lifting.

What really sets Python apart isn't just its versatility, but its origin story. It was created by a single visionary—Guido van Rossum—in the late '80s, then freely handed over to a global community that's steered its evolution ever since. No big corporation calls the shots; it's all community-driven. The license? The super-permissive Python Software Foundation License (PSF), which basically lets anyone use, tweak, or share it without hassle.

In this audit, I'll break down Python through the lens of Units 1-5 from our Open Source Software course. You'll find the shell scripts I whipped up and tested on Ubuntu in the attached GitHub repo.

Part A — Origin and Philosophy (Units 1 & 2)

A1. The Problem Python Was Created to Solve

To understand why Python was born, rewind to the late 1980s programming scene. At the CWI research institute in the Netherlands, folks built ABC—a super clean language meant for teaching newbies to code. Guido van Rossum, a developer there, helped out on it.

He loved ABC's readable syntax, but man, it had limits. No way to extend it, weak error handling, and it barely talked to the OS. Great for beginners, sure, but once you leveled up? Dead end.

Then you had the beasts: C, C++, Fortran. Total powerhouses for big jobs, but steep learning curves, no garbage collection, and a nightmare for quick scripts. Shell scripting worked for simple stuff, but anything complex? Forget it. Devs craved ABC's ease with C's muscle—and nothing filled that gap.

Guido stepped up. Over Christmas break in 1989, bored and tinkering, he started a new interpreter. Named it Python after Monty Python's Flying Circus, because why not make coding fun? By February 1991, version 0.9.0 hit Usenet, and the community jumped on it.

His genius stroke? Tossing it open for contributions right away. That sparked its community-fueled growth. The vibe—readability first, keep it simple, be explicit—got codified in Tim Peters' PEP 20, aka "The Zen of Python."

Fast-forward to 2001: the Python Software Foundation (PSF) launches to lock in its freedom forever. Guido played "Benevolent Dictator for Life" (BDFL) until 2018, then handed reins to a steering council. It's the ultimate open-source origin story—one guy's frustration turns into a global powerhouse.

A2. The Licence — What the PSF Licence Actually Says

Python runs on the Python Software Foundation License (PSF)—a super chill, permissive open-source license that's endorsed by both the Free Software Foundation and Open Source Initiative. If you've seen MIT or BSD licenses, this one's right in that family: minimal rules, maximum freedom.

It nails the four core open-source freedoms: run it however you want, peek inside and tweak the code, share copies freely, and pass on your mods. Hobby project on a rainy weekend? Billion-dollar enterprise beast? No barriers.

Key difference from GPL: PSF isn't "copyleft." Big deal. A company can grab Python, bolt it into their closed-source product, tweak it silly, and sell it—without spilling their secrets. Smart move by the founders; they bet on widespread use over mandating everything stays open. That's why giants like Google embed it everywhere, and distros like Anaconda thrive, all without legal drama.

Dimension	PSF Licence (Python)	GPL v2 (e.g. Git, Linux)
Copyleft	None. Modified code can be kept private.	Strict. If you modify and distribute, you must share the source.
Commercial use	100% allowed without sharing source code.	Allowed, but derivative works must remain open.
Patent rights	Not explicitly detailed.	Implied protection exist.
Compatibility	\Plays nicely with almost all other licenses.	Incompatible with some other licences
Best suited for	Widespread corporate and public adoption.	Guaranteeing software remains free forever.

Python nails the classic distinction: "free as in beer" (zero cost) versus "free as in liberty" (total freedom). Companies can tweak it for their proprietary stuff, so the end product might download for free—but users can't peek under the hood or remix it.

For a language like Python, the PSF license is genius; it invites *everyone* to jump in. But if I were crafting something critical, like a new OS kernel for public infra, I'd go GPL to block corporate lock-in.

A3. The Ethics of Open Source

Python's wild success raises a big question: Should all software be open source? It powers hospital scanners, banking backends, and public AI—stuff that touches millions of lives. When code runs critical systems, openness lets us audit it, catch bugs, and build trust. Closed-source black boxes in public infra? Recipe for disaster.

But mandating open source for everything would tank the commercial software world. Not realistic. Smarter path: Software in the public interest—like infrastructure tools—owes us transparency way more than your average shopping app.

Then there's the ethics mess. Volunteers and shoestring teams keep Python's powerhouse libs (NumPy, Pandas) alive. Trillion-dollar corps rake in billions using them, yet give back peanuts. When a key maintainer burns out? The ripple hits everyone. Groups like the PSF help with funding, but the give-take imbalance is glaring.

Part B — Linux Footprint on Kali Ubuntu (Unit 2)

Installation

If you are running Ubuntu, Python 3 is already baked into the OS. You can manage it directly through the advanced packaging tool (APT):

```
python3 --version          # Check installed version
sudo apt update && sudo apt install python3 python3-pip -y  # Install/update
pip3 --version             # Confirm pip package manager
```

While you can compile Python from scratch by grabbing the tarball from python.org (which requires packages like build-essential and libssl-dev, the standard APT method is perfectly sufficient for almost all academic and professional use cases.

Core Directories

Path	Purpose / Contents
<code>/usr/bin/python3</code>	The primary executable binary.
<code>/usr/lib/python3.x/</code>	Standard library modules — os, sys, json, re, etc.
<code>/usr/lib/python3/dist-packages/</code>	System-wide third-party packages installed via APT
<code>/usr/local/lib/python3.x/</code>	User-compiled or pip-installed packages (system-wide)
<code>~/.local/lib/python3.x/</code>	Per-user pip-installed packages (--user flag)
<code>/etc/python3/</code>	Global configuration settings.
<code>~/.config/pip/</code>	Per-user pip configuration and cache settings

Security and Permission

Python scripts run with whatever permissions you give them. The main binary at `/usr/bin/python3`? Owned by root, typically 755 perms—anyone can execute it, but only admins touch the file itself.

Sounds convenient, but it's a security minefield. Fire up a buggy or malicious script with `sudo`, and boom—it can trash your whole system. Linux 101: Stick to unprivileged users for scripts. Running a web app like Django? Set up a dedicated account (e.g., `www-data`). If hackers slip in, they're sandboxed—no root escalations.

Part C — The FOSS Ecosystem (Units 3 & 4)

Dependencies — What Python Is Built On

Library / Tool	Role in Python
GCC (C Compiler)	The standard version of Python (CPython) is literally written in C and compiled by GCC.
zlib	Handles all the heavy lifting for data compression in modules like <code>gzip</code> .
OpenSSL	The backbone of the <code>ssl</code> module, allowing Python to make secure HTTPS connections.
SQLite (public domain)	Provides the built-in database engine for the <code>sqlite3</code> module.
GNU readline	Makes the interactive terminal (REPL) usable, allowing you to use the arrow keys to fetch past commands.

The Ecosystem It Created

Python's no-restrictions license sparked a third-party gold rush. PyPI now boasts over 500,000 packages—insane.

- **Science & Data:** NumPy, pandas, SciPy revolutionized how researchers crunch numbers.
- **Machine Learning:** PyTorch and TensorFlow put AI in everyone's hands.
- **Web:** Django and Flask power behemoths like Instagram and Pinterest.

None of this explosion happens under a paywall. It's pure open-source magic: free to see, fork, and build on, layering innovation on innovation.

Python in the Modern Server Stack

Back in the day, LAMP (Linux, Apache, MySQL, PHP) ruled the web. Now? Python often swaps in for PHP. Think Flask or Django for backend smarts, PostgreSQL for data, Nginx as reverse proxy. Everything gets Dockerized and cloud-hosted—bye-bye bare-metal LAMP relics.

Community and Governance

The Python Software Foundation (PSF), a US-based non-profit, keeps things humming. Day-to-day? Handled by a five-person Steering Council, voted in yearly by core devs. They steer the main repo on GitHub: github.com/python/cpython.

Big changes go through Python Enhancement Proposals (PEPs)—numbered docs anyone can submit. Debates are public, decisions transparent. Standouts: PEP 572 (walrus operator `:`), PEP 20 ("The Zen of Python"), and PEP 8 (style guide).

Part D — Open Source vs Proprietary (Critical Analysis)

Python vs MATLAB — Comparison Table

Dimension	Python (Open Source — PSF Licence)	MATLAB (Proprietary — MathWorks)
Cost	Completely free. No licencing fees for any use case.	Expensive per-seat licences. Student editions subsidised but not free. Enterprise licences cost thousands per year.
Support	Massive community via Stack Overflow, PyPI, GitHub Issues, PyCon. No SLA.	Paid MathWorks technical support with guaranteed response times and official documentation.
Freedom to modify	Full freedom — fork, study, modify, redistribute under PSF Licence.	None — source code cannot be read, changed, or redistributed.
Community control	PSF-governed, community-elected steering council, transparent PEP process.	MathWorks-governed. Direction follows corporate product strategy.
Ecosystem	500,000+ packages on PyPI. Runs everywhere — Linux, Windows, macOS, ARM, cloud.	Rich toolboxes for specific domains (Simulink, Control Toolbox). Tightly integrated but proprietary.
Verdict	The undeniable choice for 95% of modern engineering and data projects.	Only necessary if you are trapped in a legacy system that heavily relies on Simulink.

Conclusion of Comparison

For most real-world projects, Python wins hands-down. It's cheaper and flat-out better than MATLAB for anything new—thanks to its killer packages, huge global community, and no license fees. MATLAB's only edge? Legacy setups like Simulink models in control engineering, where migration costs outweigh the perks. Even there, SciPy, python-control, and Simulink converters are closing the gap fast.

Hands down, I'd pick Python for data science, web dev, automation, or research—and I'd jump in to help. It's newbie-friendly too: CPython's dev guide is a breeze, GitHub has legit "easy" issues for beginners, and PEPs let anyone pitch language changes. With tens of millions of users worldwide, even doc tweaks or tests make a real dent.

5. Shell Script Documentation

All five scripts are tested on Kali Linux and submitted as .sh files in the GitHub repository. Add screenshots of each script running where indicated.

Script 1 — System Identity Report

Displays a formatted system welcome screen showing kernel, distro, user, uptime, date, and the OS licence.

Concepts: variables, echo, command substitution \$(), formatted output.

```
#!/bin/bash
# Script 1: System Identity Report
# Author: [Your Name] | Reg No: [Your Reg No] | Course: Open Source Software
# Description: Displays a formatted system welcome screen with OS and user info

STUDENT_NAME="[Your Name]"
SOFTWARE_CHOICE="Python"

# Gather system information using command substitution
KERNEL=$(uname -r)
DISTRO=$(cat /etc/os-release | grep PRETTY_NAME | cut -d= -f2 | tr -d "'")
USER_NAME=$(whoami)
HOME_DIR=$HOME
UPTIME=$(uptime -p)
CURRENT_DATE=$(date '+%d %B %Y, %H:%M')
PY_VERSION=$(python3 --version 2>&1)      # Python version installed

# Display formatted output
echo "=====
echo "  OPEN SOURCE AUDIT - SYSTEM IDENTITY REPORT  "
echo "=====
echo "Student      : $STUDENT_NAME"
echo "Software      : $SOFTWARE_CHOICE"
echo "-----"
echo "Kernel         : $KERNEL"
echo "Distribution: $DISTRO"
echo "User           : $USER_NAME"
echo "Home Dir       : $HOME_DIR"
echo "Uptime         : $UPTIME"
echo "Date/Time      : $CURRENT_DATE"
echo "Python         : $PY_VERSION"
echo "OS Licence     : GNU General Public License v2 (GPL v2) - Kali Linux"
echo "=====
```

```

nupur@Lenovo-ThinkPad-X1-Carbon-Gen-9:~/Downloads/Python$ chmod +x script1.sh
nupur@Lenovo-ThinkPad-X1-Carbon-Gen-9:~/Downloads/Python$ ./script1.sh
=====
OPEN SOURCE AUDIT — SYSTEM IDENTITY REPORT
=====
Student      : Nupur (248CE11579)
Software     : Python
-----
Distro       : Ubuntu 25.10
Kernel       : 6.17.0-19-generic
User         : nupur
Home Dir     : /home/nupur
Uptime       : up 33 minutes
Date         : Sunday, 28 March 2026
Time         : 18:45:20
Python       : Python 3.13.7
-----
OS License:
The Linux kernel is licensed under GPL v2.
You have the freedom to run, study, and modify,
and distribute this software freely.
-----
Python License:
Python is released under the PSF License
(Python Software Foundation License).
It is a permissive, BSD-style license that
allows free use, modification, and even
commercial distribution without sharing source
code — unlike GPL.
=====
nupur@Lenovo-ThinkPad-X1-Carbon-Gen-9:~/Downloads/Python$

```

Script 2 — FOSS Package Inspector

Checks if Python 3 is installed, retrieves version info, and uses a case statement to print a philosophy note about the package. Concepts: if-then-else, case statement, dpkg, grep.

```

#!/bin/bash
# Script 2: FOSS Package Inspector
# Author: [Your Name] | Reg No: [Your Reg No]
# Description: Checks package installation and prints an open-source philosophy note

PACKAGE="python3"

# Check installation using dpkg (Debian/Kali)
if dpkg -l $PACKAGE &>/dev/null; then
    echo "Package      : $PACKAGE is INSTALLED."
    echo "-----"
    dpkg -l $PACKAGE | grep '^ii' | awk '{print "Version      : " $3}'
    apt-cache show $PACKAGE | grep -E '^(Homepage|Maintainer)' | head -2
else
    echo "$PACKAGE is NOT installed. Run: sudo apt install $PACKAGE"
fi

echo ""
echo "--- Open Source Philosophy Note ---"

# Case statement: different philosophy note per package name
case $PACKAGE in

```

```

python3)
    echo "Python: Born from a Christmas holiday project, governed by a global
community."
    echo "Proof that a language built for humans – not machines – wins in the long
run." ;;
git)
    echo "Git: Born from proprietary revocation. Freedom by necessity." ;;
apache2|httpd)
    echo "Apache: The web server that made the open internet possible for
everyone." ;;
firefox)
    echo "Firefox: A nonprofit browser fighting to keep the web open and private."
;;
vlc)
    echo "VLC: Written by students – proof that open tools outlive corporations."
;;
*)
    echo "This is a free and open-source software package. Study its source." ;;
esac

```

```

nupur@Lenovo-ThinkPad-X1-Carbon-Gen-9:~/Downloads/Python$ chmod +x script2.sh
nupur@Lenovo-ThinkPad-X1-Carbon-Gen-9:~/Downloads/Python$ ./script2.sh
=====
FOSS PACKAGE INSPECTOR -- python3
=====

[✓] python3 is INSTALLED on this system.

--- Package Details ---
Status: install ok installed
Maintainer: Ubuntu Developers <ubuntu-devel-discuss@lists.ubuntu.com>
Version: 3.13.7-1
Original-Maintainer: Matthias Klose <doko@debian.org>
Homepage: https://www.python.org/

--- Installed Version ---
Python 3.13.7

--- Python Executable Location ---
/usr/bin/python3

--- Open Source Philosophy Note ---
Python: Guido van Rossum started Python in 1989 as a
hobby project over Christmas. He believed programming
should be accessible and readable for everyone -- and
the PSF License ensures it stays free forever.

=====
nupur@Lenovo-ThinkPad-X1-Carbon-Gen-9:~/Downloads/Python$

```

Script 3 — Disk and Permission Auditor

Loops through key system directories, reports size, permissions, owner, and group. Also checks for Python's library directory. Concepts: for loop, arrays, du, ls -ld, awk, cut.

```
#!/bin/bash
# Script 3: Disk and Permission Auditor
# Author: [Your Name] | Reg No: [Your Reg No]
# Description: Audits permissions and disk usage for important system directories

DIRS=("/etc" "/var/log" "/home" "/usr/bin" "/tmp")

echo "Directory Audit Report"
echo "-----"
printf "%-18s %-8s %-22s\n" "Directory" "Size" "Perms / Owner / Group"
echo "-----"

# Loop through each directory in the array
for DIR in "${DIRS[@]}; do
    if [ -d "$DIR" ]; then
        PERMS=$(ls -ld "$DIR" | awk '{print $1, $3, $4}')
        SIZE=$(du -sh "$DIR" 2>/dev/null | cut -f1)
        printf "%-18s %-8s %-22s\n" "$DIR" "${SIZE:-N/A}" "$PERMS"
    else
        echo "$DIR does not exist on this system."
    fi
done

# Check Python's standard library directory specifically
echo ""
echo "--- Python Library Directory Check ---"
PY_LIB=$(find /usr/lib -maxdepth 1 -name 'python3*' -type d | head -1)
if [ -d "$PY_LIB" ]; then
    echo "Python library found at: $PY_LIB"
    PERMS=$(ls -ld "$PY_LIB" | awk '{print $1, $3, $4}')
    echo "Permissions: $PERMS"
    echo "Module count: $(ls $PY_LIB | wc -l) items"
else
    echo "Python standard library directory not found."
fi
```

```

nupur@Lenovo-ThinkPad-X1-Carbon-Gen-9:~/Downloads/Python$ chmod +x script3.sh
nupur@Lenovo-ThinkPad-X1-Carbon-Gen-9:~/Downloads/Python$ ./script3.sh
=====
DISK AND PERMISSION AUDITOR -- SYSTEM REPORT
=====
Directory          Perms (Owner:Group)  Size
-----
/etc                drwxr-xr-x (root:root) 12M
/var/log            drwxrwxr-x (root:syslog) 192M
/home              drwxr-xr-x (root:root) 1.6G
/usr/bin            drwxr-xr-x (root:root) 293M
/tmp                drwxrwxrwt (root:root) 8.0K
-----

---- Python-Specific Installation Audit ----

[] Python binary    : /usr/bin/python3
  Permissions       : lrwxrwxrwx root root

[] Python lib dir   : /usr/lib
  Permissions       : drwxr-xr-x root root
  Size              : 2.8G

[] site-packages    : /usr/local/lib/python3.13/dist-packages
  Permissions       : drwxr-xr-x root root
  Size              : 4.0K
=====

nupur@Lenovo-ThinkPad-X1-Carbon-Gen-9:~/Downloads/Python$ █

```

Script 4 — Log File Analyzer

Reads a log file line by line, counts keyword occurrences, and shows the last 5 matching lines. Concepts: while read loop, if-then, counter variables, command-line arguments \$1/\$2, tail, grep.

```

#!/bin/bash
# Script 4: Log File Analyzer
# Usage: ./log_analyzer.sh /var/log/syslog [keyword]
# Author: [Your Name] | Reg No: [Your Reg No]

LOGFILE=$1
KEYWORD=${2:-"error"} # Default keyword is 'error' if not provided
COUNT=0

# Validate that a log file argument was provided
if [ -z "$LOGFILE" ]; then
    echo "Usage: $0 <logfile> [keyword]"; exit 1

```

```
fi

# Check file exists and is a regular file
if [ ! -f "$LOGFILE" ]; then
    echo "Error: $LOGFILE not found."; exit 1
fi

# Check if the file is empty
if [ ! -s "$LOGFILE" ]; then
    echo "Warning: $LOGFILE is empty. Nothing to analyse."; exit 0
fi

echo "Analysing : $LOGFILE"
echo "Keyword   : $KEYWORD"
echo "-----"

# Read the file line by line using a while-read loop
while IFS= read -r LINE; do
    # If the current line contains the keyword (case-insensitive), increment counter
    if echo "$LINE" | grep -iq "$KEYWORD"; then
        COUNT=$((COUNT + 1)) # Arithmetic expansion
    fi
done < "$LOGFILE"

echo "Keyword '$KEYWORD' found $COUNT time(s) in $LOGFILE"
echo ""

# Display the last 5 matching lines if any were found
if [ $COUNT -gt 0 ]; then
    echo "Last 5 matching lines:"
    grep -i "$KEYWORD" "$LOGFILE" | tail -5
fi
```

```
nupur@Lenovo-ThinkPad-X1-Carbon-Gen-9:~/Downloads/Python$ chmod +x script4
nupur@Lenovo-ThinkPad-X1-Carbon-Gen-9:~/Downloads/Python$ ./script4.sh
=====
LOG FILE ANALYZER — FOSS AUDIT
=====
Log File : test.log
Keyword  : "error"(case-insensitive)
-----

Total matches : 5
Keyword "error" found 5 time(s) in test.log

---- Last 5 Lines Matching "error" ----
>> echo "ERROR: Failed to load Python 3 module." > test.log
>> echo "ERROR: Network connection failed." >> test.log
>> echo "ERROR: Database locked." >> test.log
>> echo "ERROR: Memory limit exceeded." >> test.log
>> echo "ERROR: Apache service crashed." >> test.log

=====
nupur@Lenovo-ThinkPad-X1-Carbon-Gen-9:~/Downloads/Python$
```

Script 5 — Open Source Manifesto Generator

Asks the user 3 interactive questions and writes a personalised open-source philosophy statement to a .txt file. Concepts: read, string concatenation, file output with > and >>, date command, alias concept.

```
#!/bin/bash
# Script 5: Open Source Manifesto Generator
# Author: [Your Name] | Reg No: [Your Reg No]
# Note on aliases: 'alias pyrun="python3"' creates a terminal shortcut for python3.
# Aliases are defined in ~/.bashrc for persistence across sessions.

echo "=====
echo "  OPEN SOURCE MANIFESTO GENERATOR  "
echo "=====

# Gather user input interactively using read -p
read -p "1. One open-source tool you use every day: " TOOL
read -p "2. In one word, what does freedom mean to you? " FREEDOM
read -p "3. One thing you would build and share freely: " BUILD

DATE=$(date '+%d %B %Y')
OUTPUT="manifesto_$(whoami).txt" # Output filename uses current username
```

```
# Write manifesto to file - > creates/overwrites, >> appends
echo "Open Source Manifesto - $DATE" > "$OUTPUT"
echo "-----" >> "$OUTPUT"
echo "" >> "$OUTPUT"
echo "Every day I rely on $TOOL - a tool built openly and shared freely." >> "$OUTPUT"
echo "To me, freedom means $FREEDOM - and that is what open source represents." >>
"$OUTPUT"
echo "It is the right to learn from and build upon the work of those who came before."
>> "$OUTPUT"
echo "Python itself was written this way - by one person over a Christmas holiday," >>
"$OUTPUT"
echo "then gifted to the world so that millions could build on top of it." >> "$OUTPUT"
echo "One day I will build $BUILD and release it openly," >> "$OUTPUT"
echo "so someone I will never meet can stand on my shoulders, as I stood on others." >>
"$OUTPUT"
echo "That is the open source promise. That is my commitment." >> "$OUTPUT"

echo ""
echo "Manifesto saved to: $OUTPUT"
cat "$OUTPUT"
```



```
nupur@Lenovo-ThinkPad-X1-Carbon-Gen-9:~/Downloads/Python$ chmod +x script5.sh
nupur@Lenovo-ThinkPad-X1-Carbon-Gen-9:~/Downloads/Python$ ./script5.sh
```

```
=====
OPEN SOURCE MANIFESTO GENERATOR — VITyarthi
=====
```

You chose Python – a language born from the belief that code should be readable, accessible, and free for all.

Answer three questions to generate your manifesto.

You chose Python – a language born from the belief that

1. Nameword, what does 'freedom' mean to you? Liberty
3. Name one thing you would build and share freely:opering system

```
=====
Generating your Python open-source manifesto...
```

[+] Manifesto saved to: manifesto_nupur.txt

```
=====
MY OPEN SOURCE MANIFESTO — PYTHON EDITION
=====
```

Generated On : 28 March 2026

Author : nupur

I chose Python as the subject of my open-source audit because it represents everything open source stands for: A language built not for profit, but for people.

Every day, I use Python – a tool that exists because someone chose to build it openly and share it freely. In that, I see freedom and liberty. In software, that free- is what the PSF License protects: the right to run, read, modify, and distribute Python without restriction.

Inspired by this, I would build operating system and release it as open source – because Guido van Rossum did the same and it changed the world.

Open source is not just a License. It is a philosophy: that knowledge grows when it is shared, not hoarded.

— nupur, 28 March 2026

```
=====
nupur@Lenovo-ThinkPad-X1-Carbon-Gen-9:~/Downloads/Python$
```

6. Conclusion

This audit shows software as a deeply human endeavor, shaped by philosophy, community, and choices. Python wasn't born from corporate roadmaps or VC cash. One developer, frustrated during Christmas break, built it and set it free. The PSF license? A deliberate bet on adoption and openness over commercial gates—and it paid off, making Python the world's top language.

Exploring its Linux roots, ecosystem explosion, and edge over proprietary rivals like MATLAB drives home the course's big idea: Open source isn't fluffy ideology—it's infrastructure. ML breakthroughs, web dev for all, industry automation? All stacked on Python and its free packages. Grasping its origins and maintainers isn't just for pros; it's our duty.

7. References

- Python Software Foundation. Python History and License: <https://docs.python.org/3/license.html>
- Van Rossum, G. A Brief Timeline of Python: <https://python-history.blogspot.com>
- GNU Project. The Free Software Definition: <https://gnu.org/philosophy/free-sw.html>
- Open Source Initiative. The Open Source Definition: <https://opensource.org/osd>
- Peters, T. PEP 20 — The Zen of Python: <https://peps.python.org/pep-0020/>
- Python Enhancement Proposals (PEPs): <https://peps.python.org>
- PyPI — The Python Package Index: <https://pypi.org>
- SPDX Licence List — PSF Licence: <https://spdx.org/licenses/PSF-2.0.html>
- Choose a Licence — Comparison: <https://choosealicense.com/licenses/>
- Shotts, W. The Linux Command Line: <https://linuxcommand.org/tlcl.php>
- GNU Bash Manual: <https://gnu.org/software/bash/manual>
- CPython source repository: <https://github.com/python/cpython>